
UNIVERSIDAD EAFIT

Sistemas Operativos — Evaluación práctica, primer parcial

Editor de Texto CLI sobre llamadas al sistema POSIX

Documento de sustentación

Jhon Pulgarín Restrepo

[Nombre del integrante 2]

[Nombre del integrante 3]

Editor interactivo en C (C99) para Linux, sin GUI y sin concurrencia. Todo acceso a disco se hace con **open**, **read**, **write**, **lseek**, **ftruncate**, **fstat** y **close**. Equipo de tres: comandos base o **p a d q**, más **i s**, más **m y x**, más la integración en el Shell de clase.

Medellín, 14 de septiembre de 2026

1 Qué se construyó

Un editor de texto interactivo para Linux con dos interfaces (visual a pantalla completa e intérprete de línea al estilo de `ed`), 3.391 líneas de C en nueve módulos, más una categoría nueva en el Shell de clase. `Makefile` con `all`, `clean` y `test`; 68 escenarios de prueba automatizados.

Restricción de I/O: cumplida sin excepciones. El binario no contiene ninguna llamada a `fopen`, `fread`, `fwrite` o `fclose`, ni siquiera para leer `/proc/self/status` en el monitor de memoria. `printf` y `fgets` se usan sólo sobre `STDOUT` y `STDIN`, que es lo que el enunciado permite.

Tabla 1: Trazabilidad requisito → comando → syscall → implementación.

Nivel	Comando	Llamadas al sistema	Dónde
Base	o [archivo]	<code>open(O_RDWR O_CREAT, 0644)</code>	<code>archivo_abrir()</code>
Base	p [n]	<code>lseek</code> , <code>read</code> , <code>write</code> (FD 1)	<code>archivo_imprimir_linea()</code>
Base	a [texto]	<code>lseek(SEEK_END)</code> , <code>write</code>	<code>archivo_anexar()</code>
Base	d [n]	<code>read</code> , <code>lseek</code> , <code>write</code> , <code>ftruncate</code>	<code>archivo_borrar_linea()</code>
Base	q	<code>close</code> y liberación de memoria	<code>archivo_cerrar()</code>
2 pers.	i [n] [texto]	<code>read</code> a buffer <code>malloc</code> , <code>lseek</code> , <code>write</code>	<code>archivo_insertar_linea()</code>
2 pers.	s [palabra]	<code>lseek</code> y <code>read</code> línea a línea	<code>archivo_buscar()</code>
3 pers.	m	<code>fstat</code> : <code>st_size</code> , <code>st_mode</code> , <code>st_ino</code> , <code>st_mtime</code>	<code>archivo_metadatos()</code>
3 pers.	y [n]	<code>lseek</code> , <code>read</code> , <code>malloc</code> para enco- lar	<code>portapapeles_agregar()</code>
3 pers.	x [n]	inserción repetida desde el por- tapapeles	<code>comando_pegar()</code>
Shell	editor	<code>access</code> , <code>tcgetattr</code> , <code>fork</code> , <code>execv</code> , <code>waitpid</code> , <code>tcsetattr</code>	<code>cat_aplicaciones.c</code>
Shell	editor_cmd	<code>pipe</code> , <code>fork</code> , <code>dup2</code> , <code>execv</code> , <code>write</code> , <code>close</code> , <code>waitpid</code>	<code>cat_aplicaciones.c</code>

Se añadieron `y1` (listar portapapeles) y `yc` (vaciarlo): el requisito pide *gestión* de un portapapeles secuencial, y sin poder inspeccionarlo sería una caja negra imposible de verificar.

2 Arquitectura

Nueve módulos en tres capas. Todo lo que toca al kernel está confinado en la capa de sistema, de modo que auditar la restricción de I/O se reduce a revisar un archivo.

3 Decisión 1: un despachador, dos interfaces

El enunciado pide un editor por comandos; nosotros ya teníamos uno visual a pantalla completa. En vez de escribir dos programas, escribimos **una sola implementación de cada comando y dos formas de invocarla**: el despachador `cli_ejecutar_comando()` recibe siempre una cadena («d 2»). El modo CLI se la pasa tal cual; el modo visual la construye desde la posición del cursor (`Ctrl+D` en la línea 3 arma la cadena "d 3" y la despacha).

La interfaz la decide el kernel, salvo que `-c/-cli` la fuerce:

```
// --- CALL SYSTEM: isatty() ---
// Pregunta al kernel si el descriptor 0 esta conectado a un TTY.
if (forzar_cli || !isatty(STDIN_FILENO)) return cli_repl(ruta);
terminal_activar_modos_raw();
```

Por qué así:

- **Cero duplicación.** El borrado existe una vez, en `archivo.c`. Dos programas separados habrían envejecido por separado.

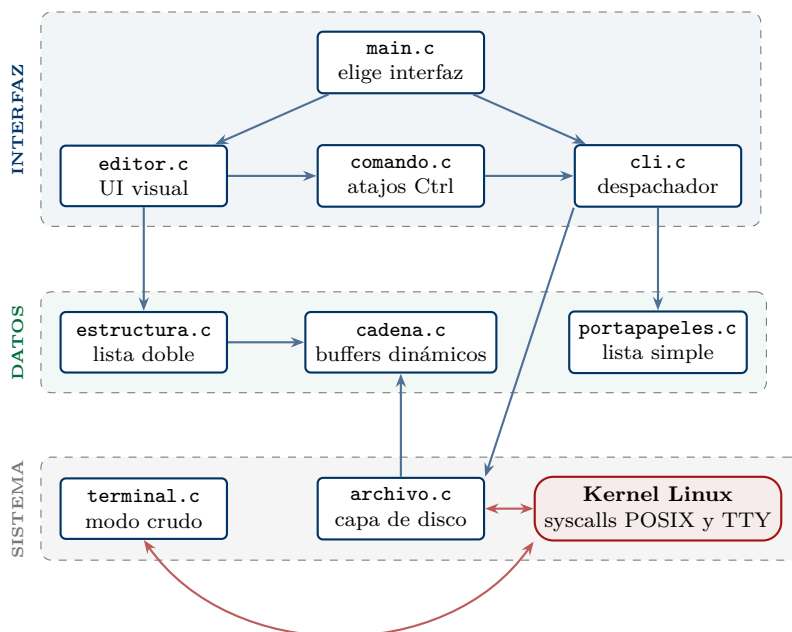


Figura 1: Sólo `archivo.c` y `terminal.c` dialogan con el kernel.

- **Automatizable.** `printf 'a hola\np\nq\n' | ./editor demo.txt` entra solo al intérprete. Ésta es la consecuencia que no habíamos previsto y que resultó decisiva: es lo que después permitió que el shell guionizara el editor **sin añadirle ni una bandera** (§7).

Dos detalles: el prompt libre va en `Ctrl+L` y no en «:» porque el editor no tiene modos y dedicar esa tecla impediría escribir dos puntos en el texto; `p` va en `Ctrl+V` porque `Ctrl+P` ya era el monitor de memoria y `Ctrl+M/I/J` llegan de la terminal como `ENTER` y `TAB`.

4 Decisión 2: el disco es la fuente de verdad

Conviven dos representaciones del documento: la lista doblemente enlazada en RAM (`estructura.c`) es la **vista**, y el descriptor con su índice de líneas (`archivo.c`) es la **fuentes de verdad**. La regla: los comandos escriben en el disco y después la vista se recarga desde el disco.

```
Ctrl+L -> d 2
1. Si habia cambios sin guardar, se persisten primero
2. archivo_borrar_linea() aplica el cambio con lseek/write/ftruncate
3. archivo_indexar() reconstruye el indice de offsets
4. editor_recargar_desde_disco() vuelve a leer la lista enlazada
```

Por qué en este sentido y no al revés (cargar en RAM y volcar al salir, que es más cómodo de programar):

- **Vaciaría el ejercicio.** El enunciado asocia `d [n]` a `lseek` y `ftruncate`. Si el documento sólo viviera en RAM, el borrado sería un juego de punteros y esas llamadas no se ejercitarían.
- **No hay estados divergentes.** Mantener RAM y disco en paralelo duplica la lógica y multiplica los casos borde; recargar los elimina como clase.
- **Lo que se ve es lo que hay en el disco**, y el banco de pruebas lo comprueba byte a byte.

El coste es releer tras cada comando: despreciable en texto plano, y el índice de offsets lo reduce a un `lseek` directo.

5 Estructuras de datos

Tres estructuras dinámicas, todas con `malloc/realloc/free`.

1. **Lista doblemente enlazada de líneas** (`estructura.c`) — el documento en RAM. Con

un vector contiguo, insertar al principio de un archivo de 50.000 líneas obligaría a desplazar todo lo demás; aquí es reconectar dos punteros. Cada `NodoLinea` contiene a su vez una lista de `NodoPalabra`, lo que abre la puerta al resaltado y a la búsqueda por palabra completa sin retokenizar. Ctrl+E vuelca los nodos con sus direcciones reales.

2. Índice de offsets (archivo.c) — el mapa del archivo en disco:

```
typedef struct {
    int      fd;                // descriptor devuelto por open()
    off_t    *inicio;          // offset absoluto donde empieza cada
    linea
    size_t    *largo;          // largo de cada linea, sin el '\n'
    int      total;
    size_t    capacidad;       // crece x2 con realloc
    int      termina_sin_salto; // 1 si el ultimo byte no es '\n'
    off_t     tamano;
} Archivo;
```

```
archivo.txt:  "hola\nmundo\n"      inicio[] = { 0, 5 }
              ^      ^              largo[]  = { 4, 5 }    (sin el salto)
```

`archivo_indexar()` recorre el archivo por bloques de 4 KB contando bytes hasta cada `\n` —el recorrido que el enunciado describe para `p`— y se reconstruye tras cada mutación. Convierte `p 7` en un `lseek` directo, y le da a `d [n]` los dos números que necesita para desplazar la cola y truncar.

3. Portapapeles secuencial (portapapeles.c) — lista enlazada simple con puntero a la cola, que conserva el orden de copiado (y 3, luego y 1, y x 5 inserta ambas en ese orden). Guarda una **copia propia** del texto: el buffer del llamador se libera al terminar el comando, así que guardar el puntero sería un *use-after-free* visible sólo al pegar.

Buffers dinámicos. La primera versión armaba las líneas sobre un `char buffer[4096]` de la pila, que cualquier línea más larga corrompía. Hoy cada línea se materializa con un `malloc` del tamaño exacto y el texto de tamaño desconocido se acumula en una `Cadena` que duplica capacidad con `realloc`. Las pruebas incluyen un caso con una línea de 8.000 bytes.

6 Anatomía de `d [n]`

El enunciado pide borrar «desplazando los bytes posteriores y truncando»:

```
Archivo antes:   u n o \n d o s \n t r e s \n
                  ^      ^
inicio[1] = 4 ---+          +--- fin de la linea 2 = inicio[2] = 8

1. leer_cola(fd, desde=8)    -> malloc con "tres\n"          (read)
2. lseek(fd, 4, SEEK_SET)    (lseek)
3. write(fd, "tres\n", 5)    (write)

Archivo ahora:   u n o \n t r e s \n o s \n      <- sobran 4 bytes basura
4. ftruncate(fd, 13 - 4 = 9)    (ftruncate)

Archivo final:   u n o \n t r e s \n
```

Sin el paso 4 el archivo conservaría la basura al final. Ése es el motivo exacto por el que `ftruncate` está en la tabla del enunciado, y por el que las pruebas comparan el archivo y no sólo STDOUT: un editor que imprime bien pero deja basura pasaría cualquier prueba que sólo mire la pantalla.

```
// --- CALL SYSTEM: ftruncate() ---
// El archivo aun conserva al final los bytes viejos de la linea borrada
.
if (ftruncate(a->fd, a->tamano - (off_t)bytes_borrados) == -1) {
    perror("ftruncate");
}
```

```

    return -1;
}
return archivo_indexar(a);

```

7 Integración con el Shell: por qué una categoría nueva

Decisión: se creó la categoría **aplicaciones**.

El shell de clase es **dirigido por tabla**: cada comando es una entrada con nombre, categoría, uso, descripción, syscalls y manejador. La categoría no es decorativa —organiza **help** en sus tres modos y cada una vive en su propio `cat_*.c`—, así que es la unidad de organización del proyecto.

Tabla 2: Contrato implícito de cada categoría original.

Categoría	Comandos	Qué tienen en común
datos	<code>d_create</code> , <code>d_read</code> , <code>d_info</code> , <code>d_copy</code>	Demuestran una syscall de archivos y retornan de inmediato .
memoria	<code>m_sbrk</code> , <code>m_mmap</code> , <code>m_info</code>	Manipulan el espacio de direcciones del propio shell .
monitoreo	<code>p_fork</code> , <code>p_exec</code> , <code>p_kill</code> , <code>p_monitor</code>	Observan y manipulan procesos y señales .
utilidades	<code>saludar</code> , <code>hora</code> , <code>fecha</code>	Consultas triviales al sistema.

Alternativas descartadas

datos. Es el encaje tentador: mismas syscalls que `d_read`. Pero clasificar por *qué syscall usa* confunde la parte con el todo. Esa categoría tiene un contrato: ejecutar, imprimir su traza y devolver el prompt. El editor se queda vivo, borra la pantalla y se apodera del teclado. Quien viene de `d_info` esperaría un volcado y se encontraría una aplicación de la que no sabe salir: la categoría dejaría de describir a sus miembros.

monitoreo. El *mecanismo* que usamos —`fork`, `execv`, `waitpid`— es literalmente el de `p_exec`, pero el mecanismo no es el propósito. Esa categoría agrupa comandos *sobre procesos*, donde el proceso es el objeto de estudio; aquí es sólo el vehículo. Clasificar por mecanismo metería ahí cualquier programa externo, vaciando la categoría de significado.

No tocar el shell y usar `p_exec` `../editor notas.txt`. Funciona hoy sin escribir código, y lo dejamos documentado como salida de emergencia. Pero (1) no es integración sino coincidencia: **help** no lo lista y el usuario debe saber dónde está el binario; (2) no guarda ni repone el estado del TTY; (3) no puede alimentarle órdenes por su entrada estándar.

Incorporarlo como *builtin*. Compartirían espacio de direcciones: un desbordamiento del editor corrompería el shell y un `exit()` lo mataría entero. `fork` + `exec` da aislamiento de memoria y de descriptores, y un código de salida inspeccionable. Es el motivo por el que un shell real no enlaza `vim` dentro de sí mismo.

La decisión

aplicaciones — programas externos interactivos que el shell lanza como procesos hijo, **cediéndoles la terminal** hasta que terminan, y recuperándola después.

La razón es arquitectónica, no temática: esta clase de comando impone al shell un **protocolo que ninguna categoría existente necesita** —guardar el TTY, ceder, esperar, restaurar—. Una categoría, en este shell, es un contrato sobre cómo el shell trata al comando; el contrato es nuevo, luego la categoría también. Y generaliza: mañana un visor de logs o un depurador entrarían aquí sin discusión.

Qué se implementó

`shell/cat_aplicaciones.c` (nuevo, 386 líneas) con dos comandos:

- `editor [-c] [archivo]` — `access`, `tcgetattr`, `fork`, `execv`, `waitpid`, `tcsetattr`. El shell **no interpreta las banderas**: las reenvía tal cual, así que `editor -c` abre el intérprete de línea y `editor -help` la ayuda del editor, sin que el shell las conozca.
- `editor_cmd <archivo> «órdenes;...>` — `pipe`, `fork`, `dup2`, `execv`, `write`, `close`, `waitpid`. Le enchufa una tubería al descriptor 0 y el editor entra solo al intérprete, por la decisión de §3. No hace falta terminar las órdenes con `q`: al cerrar el extremo de escritura, el fin de archivo *es* la orden de salir.

`resolver_editor()` prueba `$EDITOR_SO`, `./editor`, `../editor` y `../editorTexto/editor` con `access(X_OK)`. Se comprueba **en el padre** para poder dar un mensaje claro, y se usa `execv` y no `execvp` para que el `PATH` del usuario no decida qué binario corre.

El problema central: la terminal es compartida

El editor deja el TTY en modo crudo con `tcsetattr`. Ese estado **no pertenece al proceso sino al terminal**, que padre e hijo comparten. Si el editor sale normalmente lo restaura con `atexit`; si muere por una señal, nadie lo hace y el shell hereda una terminal sin eco, sin `Ctrl+C` y sin línea canónica.

```
struct termios estado_terminal;
tcgetattr(STDIN_FILENO, &estado_terminal);    /* 1. guardar antes de
ceder */
/* fork + execv + waitpid */
tcsetattr(STDIN_FILENO, TCSAFLUSH, &estado_terminal); /* 2. reponer
pase lo que pase */
```

La restauración va después del `waitpid` y **fuera de todo condicional de éxito**. Cuando detectamos muerte por señal, el shell lo dice explícitamente.

Cambios sobre el shell original

Aditivos: ningún comando ni categoría original fue modificado. `cat_aplicaciones.c` (nuevo); `shell.h` (dos prototipos); `main.c` (dos entradas en `commands[]` y `print_help` acepta la categoría); `Makefile` (nuevo fuente, objetivo `compilar` para el guion de pruebas, y corregido `clean`, que borraba `$(TARGET)` —variable inexistente—, por lo que el binario nunca se eliminaba).

8 Manejo de errores

- **Toda** `syscall` comprueba su retorno y reporta con `perror()` indicando la operación entre paréntesis: `perror("lseek (borrar)")`.
- `read` y `write` pasan por `leer_todo()` y `escribir_todo()`, que reintentan ante operaciones **parciales** y ante `EINTR`. Suponer que un `write()` de N bytes escribe N bytes es un error clásico; aquí se evita por construcción.
- Todo `malloc/realloc` comprueba `NULL`.
- Los comandos validan argumentos **antes** de tocar el disco: índice fuera de rango, argumento no numérico, archivo sin abrir, portapapeles vacío.
- Una apertura fallida termina con código de salida distinto de cero.
- `q` marca una bandera y deja que el bucle de `main()` termine; *entonces* se libera la memoria y se cierra el descriptor. Una versión anterior llamaba a `exit(0)` y dejaba fugas, incumpliendo el requisito del comando.

9 Pruebas

`pruebas.sh` (`make test`) ejercita **68 escenarios**. Principio rector: verificar la salida por pantalla no basta, así que cada caso que muta el archivo compara además los bytes en disco. Cubre cada comando y requisito acumulativo, la selección de interfaz (`-c`, `-cli`, `isatty`), la integración con el shell (que `aplicaciones` aparece en `help`, que `editor_cmd` modifica los bytes

reales, que el editor termina con código 0 al cerrarse la tubería, que `EDITOR_SO` tiene prioridad, y que sin binario a la vista el shell no revienta) y los casos borde (archivo vacío, sin salto final, línea de 8 KB, índices inválidos, comandos sin archivo abierto).

Los atajos visuales también se prueban: como sólo arrancan si `isatty` ve una terminal, el script usa `script(1)` para darles un **pseudo-terminal** y les envía pulsaciones reales. Hacen falta pausas entre teclas porque cada vuelta al modo crudo usa `TCSAFLUSH`, que descarta la entrada pendiente.

10 Conclusiones

1. **Las restricciones del enunciado son el contenido, no el obstáculo.** Que el disco sea la fuente de verdad nace de tomarse en serio la prohibición de `stdio`: es la única forma de que `lseek` y `ftruncate` hagan trabajo real en cada comando.
2. **Una decisión temprana pagó tres veces.** Elegir la interfaz con `isatty` se hizo para no duplicar código; acabó permitiendo automatizar las pruebas y después guionizar el editor desde el shell sin añadir una sola bandera.
3. **Clasificar es diseñar.** La pregunta sobre la categoría parecía de etiquetado y resultó arquitectónica: obligó a definir qué es una categoría en ese shell y a descubrir que un programa interactivo impone un protocolo nuevo.
4. **La terminal es un recurso compartido.** El punto más instructivo no está en el editor sino en su frontera con el shell: un estado que no pertenece a ningún proceso y que alguien debe restaurar cuando el otro muere sin poder hacerlo.

Trabajo futuro. Deshacer/rehacer con archivos de `swap` en `/tmp` y `unlink` al cerrar, y múltiples buffers con `fcntl/F_SETLK`. La arquitectura los admite sin cambios estructurales: el undo registrando las mutaciones de `archivo.c`, y los buffers convirtiendo la estructura `Archivo` única en una tabla de descriptores.

Anexo: comandos y ejecución

Comando	Atajo	Qué hace
<code>o [archivo]</code>	<code>Ctrl+O</code>	Abre un archivo; si no existe lo crea con 0644.
<code>p / p [n]</code>	<code>Ctrl+V</code>	Imprime todo el archivo numerado, o la línea <i>n</i> .
<code>a [texto]</code>	<code>Ctrl+A</code>	Añade el texto como nueva última línea.
<code>d [n]</code>	<code>Ctrl+D</code>	Borra la línea <i>n</i> , desplaza la cola y trunca.
<code>i [n] [texto]</code>	<code>Ctrl+N</code>	Inserta como línea <i>n</i> , desplazando el resto.
<code>s [palabra]</code>	<code>Ctrl+F</code>	Busca y lista coincidencias con línea y columna.
<code>m</code>	<code>Ctrl+G</code>	Metadatos vía <code>fstat</code> : tamaño, permisos, inodo, fecha.
<code>y [n] / x [n]</code>	<code>Ctrl+Y</code> / <code>Ctrl+U</code>	Copia al portapapeles / pega desde la línea <i>n</i> .
<code>yl / yc</code>	—	Lista / vacía el portapapeles.
<code>q</code>	<code>Ctrl+Q</code>	Cierra el descriptor y sale sin fugas.
—	<code>Ctrl+L</code>	Prompt libre para cualquier comando.
—	<code>Ctrl+S</code>	Guarda la vista con <code>write</code> y reindexa.
—	<code>Ctrl+E</code> / <code>Ctrl+P</code>	Volcado de estructuras / monitor de memoria vía <code>/proc</code> .

```
make ; make test ; make clean           # compilar, probar, limpiar
./editor notas.txt                      # modo visual
./editor -c notas.txt                   # interprete de linea
printf 'a hola\np\nq\n' | ./editor notas.txt # modo automatico (
    isatty)

cd shell ; make                          # compila y arranca eafitOS
eafitOS> help aplicaciones
```

```
eafitOS> editor -c notas.txt  
eafitOS> editor_cmd notas.txt "a hola; p; m"
```

Nota del Makefile: las macros `_DEFAULT_SOURCE` y `_POSIX_C_SOURCE=200809L` no son decorativas. Con `-std=c99` el compilador activa `__STRICT_ANSI__` y glibc esconde los prototipos POSIX que el proyecto necesita (`ftruncate`, `fstat`, `isatty`, `TIOCGWINSZ`): el estándar del lenguaje y el del sistema operativo son dos cosas distintas.